

Prof. Denilson Alves Pereira

Carlos Eduardo Oliveira Ferreira Santos

Leonardo Carvalho Silva

Matheus Henrique de Oliveira

Matheus Vítor Ferreira

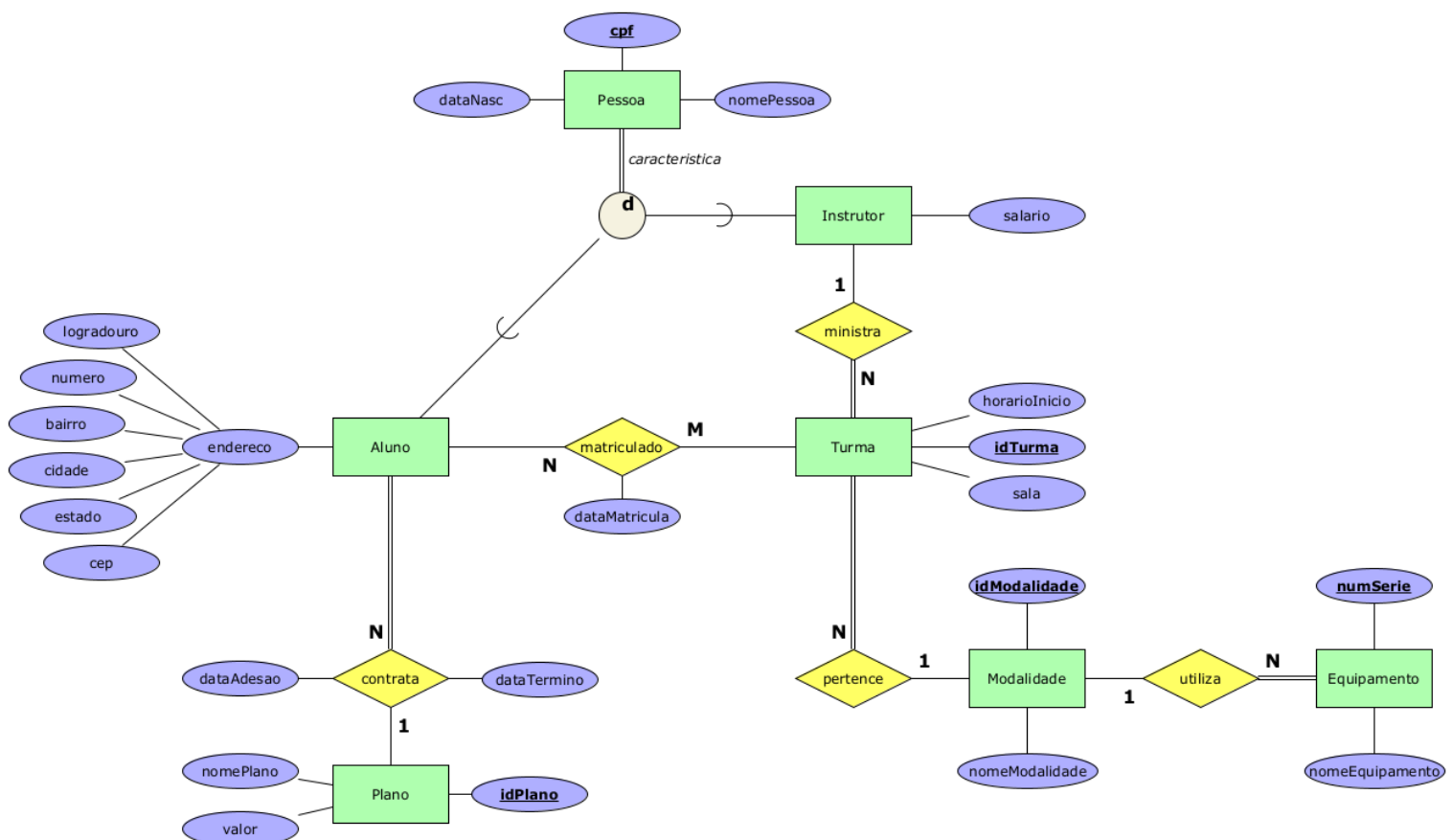
Thales Maia Mendonça Negrão

Período: 2026/1

Descrição

Levantamento de dados para o sistema de gestão de uma academia de ginástica. A academia possui um conjunto de pessoas, centralizadas na superclasse Pessoa, da qual é importante armazenar os atributos cpf, nomePessoa e dataNasc. As pessoas são especializadas em Aluno ou Instrutor. Dos indivíduos do tipo Instrutor, é necessário armazenar o salário mensal. Para os do tipo Aluno, deve-se registrar o endereço completo, detalhado em logradouro, numero, bairro, cidade, estado e cep. A academia oferece opções de Plano, as quais possuem um idPlano, um nomePlano e um valor. Cada Aluno contrata um plano, sendo fundamental registrar a dataAdesao e a dataTermino correspondente. As atividades são organizadas por Modalidade, identificadas por idModalidade e nomeModalidade. Cada modalidade dá origem a diversas Turma, que possuem idTurma, horariolnicio e sala. Toda turma é ministrada por um Instrutor. Os Aluno podem estar matriculados em várias turmas, sendo necessário saber a dataMatricula de cada um. Por fim, o sistema controla o inventário de Equipamento, onde cada item possui um numSerie e um nomeEquipamento, sendo vinculado à Modalidade que o utiliza.

Diagrama ER



Dicionário de Dados - ER

Tipo Entidade: Aluno

Descrição: Subclasse de Pessoa que representa os clientes matriculados. Armazena dados de localização divididos em seis atributos atômicos para controle de endereço postal.

Nome	Descrição	Domínio	Permite nulo?
logradouro	Nome da via pública.	Texto(50)	N
numero	Identificação numérica da residência.	Inteiro(5) positivo	N
bairro	Bairro de residência do aluno.	Texto(30)	S
cidade	Cidade de residência.	Texto(30)	N
estado	Sigla da unidade federativa.	Texto(2)	N
cep	Código de endereçamento postal.	Texto(8)	N

Tipo Entidade: Equipamento

Descrição: Tipo Entidade que representa o inventário físico da academia. Cada item possui um numSerie único e é vinculado a uma Modalidade específica para controle de uso.

Nome	Descrição	Domínio	Permite nulo?
numSerie	Identificador único de fábrica.	Texto(20)	N
nomeEquipamento	Nome comercial da máquina.	Texto(50)	N

Tipo Entidade: Instrutor

Descrição: Subclasse de Pessoa referente aos profissionais de educação física. Responsável por ministrar aulas e possui o atributo específico de salário mensal.

Nome	Descrição	Domínio	Permite nulo?
salario	Remuneração mensal fixa do instrutor.	Real(8,2) positivo	N

Tipo Entidade: Modalidade

Descrição: Categoria de atividade física oferecida pela academia. Define os tipos de exercícios que utilizam equipamentos específicos e originam as diferentes turmas do sistema.

Nome	Descrição	Domínio	Permite nulo?
idModalidade	Código identificador da modalidade.	Inteiro(3) positivo	N
nomeModalidade	Nome da atividade física.	Texto(40)	N

Tipo Entidade: Pessoa

Descrição: Superclasse que centraliza os dados básicos de identificação como cpf, nomePessoa e dataNasc, servindo de base para a generalização de usuários.

Nome	Descrição	Domínio	Permite nulo?
cpf	Cadastro de Pessoa Física.	Texto(11)	N
nomePessoa	Nome completo do indivíduo.	Texto(80)	N
dataNasc	Data de nascimento.	Data	N

Tipo Entidade: Plano

Descrição: Define as opções de contrato financeiro disponíveis para os alunos. Contém informações sobre o valor da adesão e o nome descritivo do pacote de serviços oferecido.

Nome	Descrição	Domínio	Permite nulo?
idPlano	Identificador único do plano comercial.	Inteiro(3) positivo	N
nomePlano	Nome do pacote (ex: VIP, Mensal).	Texto(30)	N
valor	Preço bruto da adesão ao plano.	Real(7,2) positivo	N

Tipo Entidade: Turma

Descrição: Instância operacional de uma Modalidade em horário e local definidos. É obrigatoriamente vinculada a um Instrutor e pode possuir múltiplos alunos matriculados.

Nome	Descrição	Domínio	Permite nulo?
idTurma	Identificador único da turma.	Inteiro(5) positivo	N
horarioInicio	Horário previsto para início da aula.	Hora	N
sala	Identificação do local físico da aula.	Texto(20)	N

Tipo Relacionamento: contrata

Descrição: Associa Aluno a um Plano. Registra a dataAdesao, definindo o início da vigência contratual e as permissões de acesso do cliente.

Nome	Descrição	Domínio	Permite nulo?
dataAdesao	Data em que o contrato foi assinado.	Data	N
dataTermino	Data do fim do contrato	Data	N

Tipo Relacionamento: matriculado

Descrição: Vincula Aluno a Turma. Armazena a dataMatricula, permitindo o controle de lotação das salas e a gestão de frequência nas aulas oferecidas.

Nome	Descrição	Domínio	Permite nulo?
dataMatricula	Data de ingresso do aluno na turma.	Data	N

Tipo Relacionamento: ministra

Descrição: Define a responsabilidade técnica do Instrutor sobre a Turma. Estabelece o vínculo entre o profissional e o horário de aula específico.

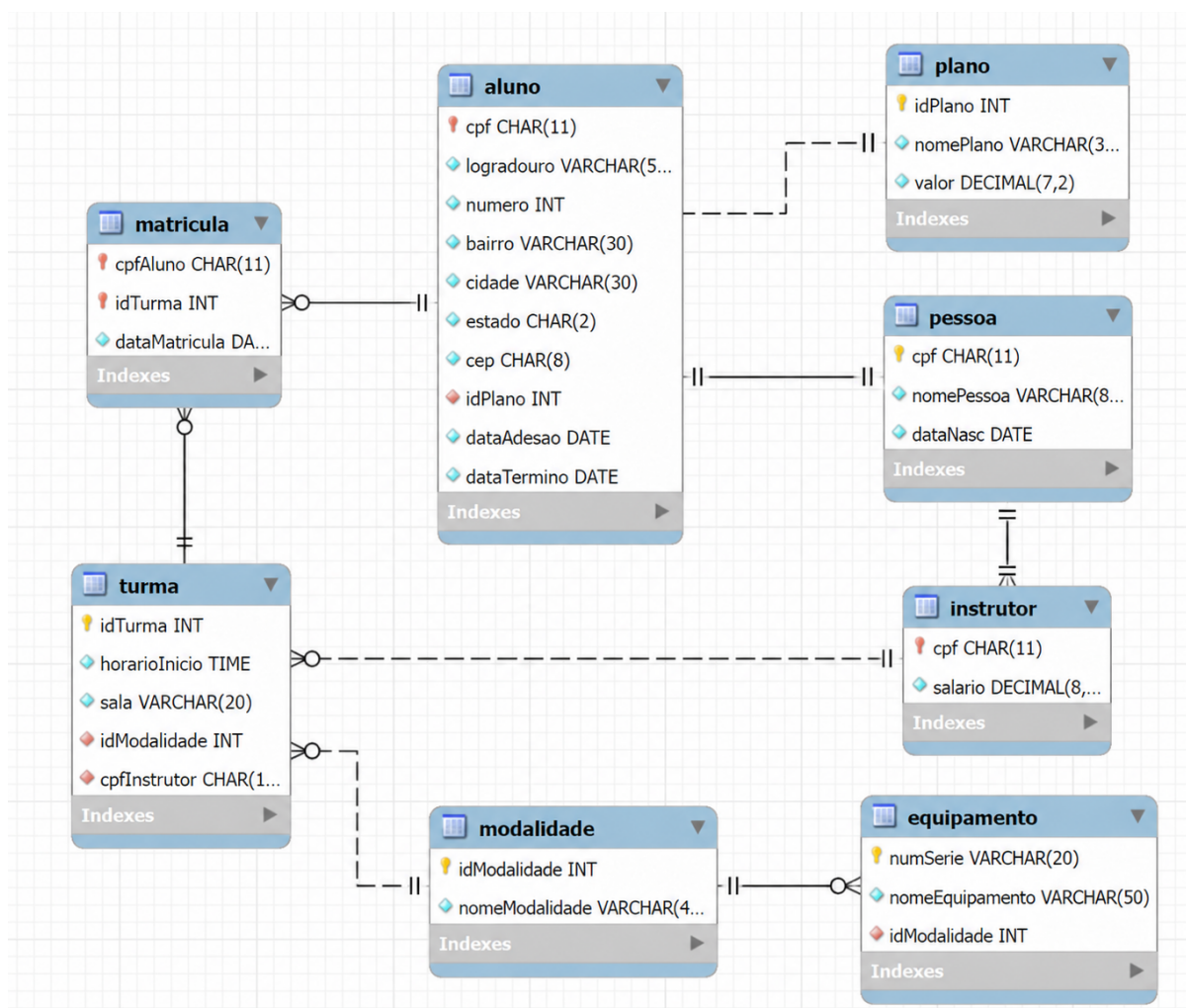
Tipo Relacionamento: pertence

Descrição: Relaciona a Turma à sua Modalidade de origem. Garante a correta classificação das aulas conforme o programa de exercícios da categoria.

Tipo Relacionamento: utiliza

Descrição: Indica quais itens de Equipamento são necessários para uma Modalidade. Mapeia a infraestrutura física indispensável para a prática da atividade.

Diagrama Relacional



Dicionário de Dados Relacional

Tabela: Pessoa

Descrição: Superclasse que centraliza os dados básicos de identificação para todos os indivíduos vinculados à academia.

Nome	Descrição
cpf	Cadastro de Pessoa Física. Identificador único e chave primária.
nomePessoa	Nome completo do indivíduo.
dataNasc	Data de nascimento do indivíduo.

Tabela: Plano

Descrição: Define as opções de contrato financeiro e pacotes de serviços disponíveis para adesão.

Nome	Descrição
idPlano	Identificador único do plano comercial. Chave primária.
nomePlano	Nome descritivo do pacote.
valor	Preço bruto da adesão ao plano.

Tabela: Aluno

Descrição: Subclasse de **Pessoa** representando os clientes, contendo dados de endereço e o vínculo com o plano contratado.

Nome	Descrição
cpf	Identificador do aluno. Chave primária e chave estrangeira referenciando Pessoa. Opção de exclusão: propagação (cascade).
logradouro	Nome da via pública de residência.
numero	Identificação numérica da residência.
bairro	Bairro de residência do aluno (permite nulo).
cidade	Cidade de residência.
estado	Sigla da unidade federativa.
cep	Código de endereçamento postal.
idPlano	Referência ao Plano contratado. Chave estrangeira. Opção de exclusão: bloqueio (restrict).
dataAdesao	Data em que o contrato do plano foi assinado.
dataTermino	Data prevista para o fim da vigência do contrato.

Tabela: Instrutor

Descrição: Subclasse de **Pessoa** referente aos profissionais de educação física da academia.

Nome	Descrição
cpf	Identificador do instrutor. Chave primária e chave estrangeira referenciando Pessoa. Opção de exclusão: propagação (cascade).
salario	Remuneração mensal fixa do profissional.

Tabela: Modalidade

Descrição: Categorias de atividades físicas oferecidas que agrupam turmas e equipamentos.

Nome	Descrição
idModalidade	Código identificador da modalidade. Chave primária.
nomeModalidade	Nome da atividade física.

Tabela: Turma

Descrição: Instâncias operacionais de uma modalidade em horários e locais definidos sob responsabilidade de um instrutor.

Nome	Descrição
idTurma	Identificador único da turma. Chave primária.
horarioInicio	Horário previsto para início da aula.
sala	Identificação do local físico da aula.
idModalidade	Referência à Modalidade de origem. Chave estrangeira. Opção de exclusão: bloqueio (restrict).
cpfInstrutor	Referência ao Instrutor que ministra a aula. Chave estrangeira. Opção de exclusão: bloqueio (restrict).

Tabela: Equipamento

Descrição: Itens do inventário físico vinculados às modalidades que os utilizam.

Nome	Descrição
numSerie	Identificador único de fábrica. Chave primária.
nomeEquipamento	Nome comercial da máquina.
idModalidade	Referência à Modalidade que utiliza o item. Chave estrangeira. Opção de exclusão: bloqueio (restrict).

Tabela: Matricula

Descrição: Tabela associativa que registra o vínculo de alunos em turmas específicas.

Nome	Descrição
cpfAluno	Referência ao Aluno. Chave primária composta e chave estrangeira. Opção de exclusão: propagação (cascade).
idTurma	Referência à Turma. Chave primária composta e chave estrangeira. Opção de exclusão: propagação (cascade).
dataMatricula	Data de ingresso do aluno na turma correspondente.

ETAPA 3: Scripts do banco de dados

```
--
=====
=
-- TRABALHO PRATICO - ETAPA 3 - IMPLEMENTACAO EM SQL
-- GCC214 - Introducao a Sistemas de Banco de Dados - UFLA - 2026/1
-- Grupo 07 - Sistema de Gestao de Academia de Ginastica
--
#####
#
-- (a) CRIACAO DE TABELAS E RESTRICOES DE INTEGRIDADE
--     PK, FK em todas as tabelas + exemplos de UNIQUE e DEFAULT
--
#####
#

-- Apaga o banco se existir e cria novamente com charset utf8mb4.
DROP DATABASE IF EXISTS academia;
CREATE DATABASE academia CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE academia;

-- Pessoa: superclasse. Guarda os dados comuns a alunos e
instrutores.
-- PRIMARY KEY em cpf.
```



```

CREATE TABLE Pessoa (
    cpf          CHAR(11)          NOT NULL,
    nomePessoa   VARCHAR(80)       NOT NULL,
    dataNasc     DATE              NOT NULL,
    CONSTRAINT pk_pessoa PRIMARY KEY (cpf)
);

-- Plano: tipos de plano da academia.
-- Exemplo de UNIQUE: dois planos nao podem ter o mesmo nome.
-- CHECK garante id e valor positivos.
CREATE TABLE Plano (
    idPlano      INT               NOT NULL,
    nomePlano    VARCHAR(30)       NOT NULL,
    valor        DECIMAL(7,2)      NOT NULL,
    CONSTRAINT pk_plano PRIMARY KEY (idPlano),
    CONSTRAINT uk_plano_nome UNIQUE (nomePlano),          --
EXEMPLO DE UNIQUE
    CONSTRAINT ck_plano_id CHECK (idPlano > 0),
    CONSTRAINT ck_plano_valor CHECK (valor > 0)
);

-- Modalidade: modalidades oferecidas (musculacao, pilates, etc.).
-- idModalidade AUTO_INCREMENT (gerado pelo MySQL) - usado tambem
pela
-- interface Web. Exemplo de UNIQUE no nome da modalidade.
CREATE TABLE Modalidade (
    idModalidade INT PRIMARY KEY NOT NULL AUTO_INCREMENT,
    nomeModalidade VARCHAR(40) NOT NULL,
    CONSTRAINT uk_modalidade_nome UNIQUE (nomeModalidade) -- EXEMPLO
DE UNIQUE
);

-- Aluno: subclasse de Pessoa (heranca 1:1). cpf e PK e FK ao mesmo
tempo.
-- Exemplo de DEFAULT: estado assume 'MG' e cidade 'Lavras' quando
omitidos.
-- bairro e o unico atributo que aceita NULL.
-- FK para Plano com ON DELETE RESTRICT (nao deixa apagar plano em
uso).
CREATE TABLE Aluno (
    cpf          CHAR(11)          NOT NULL,
    logradouro    VARCHAR(50)       NOT NULL,

```

```

    numero        INT                NOT NULL,
    bairro        VARCHAR(30)        NULL,
    cidade        VARCHAR(30)        NOT NULL DEFAULT 'Lavras', --
EXEMPLO DE DEFAULT
    estado        CHAR(2)            NOT NULL DEFAULT 'MG',      --
EXEMPLO DE DEFAULT
    cep           CHAR(8)            NOT NULL,
    idPlano       INT                NOT NULL,
    dataAdesao    DATE               NOT NULL,
    dataTermino   DATE               NOT NULL,
    CONSTRAINT pk_aluno PRIMARY KEY (cpf),
    CONSTRAINT ck_aluno_numero CHECK (numero > 0),
    CONSTRAINT fk_aluno_pessoa FOREIGN KEY (cpf)
        REFERENCES Pessoa(cpf)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_aluno_plano FOREIGN KEY (idPlano)
        REFERENCES Plano(idPlano)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

-- Instrutor: subclasse de Pessoa (heranca 1:1). cpf PK e FK.
CREATE TABLE Instrutor (
    cpf           CHAR(11)           NOT NULL,
    salario       DECIMAL(8,2)       NOT NULL,
    CONSTRAINT pk_instrutor PRIMARY KEY (cpf),
    CONSTRAINT ck_instrutor_salario CHECK (salario > 0),
    CONSTRAINT fk_instrutor_pessoa FOREIGN KEY (cpf)
        REFERENCES Pessoa(cpf)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- Turma: aula de uma modalidade, ministrada por um instrutor.
-- Duas FKs obrigatorias (modalidade e instrutor), ambas RESTRICT.
CREATE TABLE Turma (
    idTurma       INT                NOT NULL,
    horarioInicio TIME               NOT NULL,
    sala          VARCHAR(20)        NOT NULL,
    idModalidade  INT                NOT NULL,
    cpfInstrutor  CHAR(11)           NOT NULL,

```

```

CONSTRAINT pk_turma          PRIMARY KEY (idTurma),
CONSTRAINT ck_turma_id       CHECK (idTurma > 0),
CONSTRAINT fk_turma_modalidade FOREIGN KEY (idModalidade)
    REFERENCES Modalidade(idModalidade) ON DELETE RESTRICT,
CONSTRAINT fk_turma_instrutor FOREIGN KEY (cpfInstrutor)
    REFERENCES Instrutor(cpf)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

-- Equipamento: equipamentos vinculados a uma modalidade.
CREATE TABLE Equipamento (
    numSerie          VARCHAR(20)    NOT NULL,
    nomeEquipamento  VARCHAR(50)    NOT NULL,
    idModalidade      INT             NOT NULL,
    CONSTRAINT pk_equipamento        PRIMARY KEY (numSerie),
    CONSTRAINT fk_equipamento_modalidade FOREIGN KEY (idModalidade)
        REFERENCES Modalidade(idModalidade)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

-- Matricula: tabela associativa N:M entre Aluno e Turma.
-- PK composta (cpfAluno, idTurma); as duas colunas sao FK
(CASCADE).
CREATE TABLE Matricula (
    cpfAluno          CHAR(11)    NOT NULL,
    idTurma           INT          NOT NULL,
    dataMatricula     DATE         NOT NULL,
    CONSTRAINT pk_matricula        PRIMARY KEY (cpfAluno, idTurma),
    CONSTRAINT fk_matricula_aluno  FOREIGN KEY (cpfAluno)
        REFERENCES Aluno(cpf)      ON DELETE CASCADE,
    CONSTRAINT fk_matricula_turma  FOREIGN KEY (idTurma)
        REFERENCES Turma(idTurma)  ON DELETE CASCADE
);

```

B)

```

--
#####
#
-- CRIAÇÃO DA TABELA PARA EXEMPLOS
--
#####
#

CREATE TABLE Funcionarios (
    idFuncionario INT NOT NULL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
    cargo VARCHAR(30),
    salario DECIMAL(10,2),
    dataAdmissao DATE
);

--
#####
#
-- ALTER TABLE (3 exemplos diversos)
--
#####
#

-- 1. ADICIONAR COLUNA: Inclui um campo para o e-mail corporativo.
ALTER TABLE Funcionarios ADD COLUMN email VARCHAR(100);

-- 2. MODIFICAR COLUNA: Aumenta a capacidade do campo 'cargo' e
define um valor padrão.
ALTER TABLE Funcionarios MODIFY cargo VARCHAR(60) DEFAULT
'Colaborador';

-- 3. ADICIONAR RESTRICAO: Garante que o e-mail não se repita na
tabela.
ALTER TABLE Funcionarios ADD CONSTRAINT uk_email_unico UNIQUE
(email);

--
#####
#
-- DROP TABLE

```

```
--
#####
#

-- Exclui a tabela após realizar as modificações e testes.
DROP TABLE Funcionarios;
```

c)

```
--
#####
#

-- (c) INSERCOES (pelo menos 5 por tabela)
--     Insercoes feitas com lista de colunas explicita, respeitando
a
--     ordem das chaves estrangeiras.
--
#####
#

-- Pessoa (16 registros: 5 viram instrutores, 11 viram alunos)
INSERT INTO Pessoa (cpf, nomePessoa, dataNasc) VALUES
('11111111111', 'Ana Souza',      '1985-03-12'),
('22222222222', 'Bruno Lima',     '1990-07-25'),
('33333333333', 'Carla Mendes',   '1988-11-02'),
('44444444444', 'Diego Rocha',    '1992-01-30'),
('55555555555', 'Eduarda Pinto',   '1983-06-18'),
('60000000001', 'Felipe Alves',   '2000-02-14'),
('60000000002', 'Gabriela Dias',  '1999-09-09'),
('60000000003', 'Henrique Costa', '2001-12-05'),
('60000000004', 'Isabela Nunes',  '1997-04-21'),
('60000000005', 'Joao Pereira',   '1995-08-17'),
('60000000006', 'Karina Santos',  '2002-03-03'),
('60000000007', 'Lucas Martins',  '1998-10-28'),
('60000000008', 'Mariana Oliveira', '2000-05-11'),
('60000000009', 'Nicolas Ferreira', '1996-07-07'),
('60000000010', 'Olivia Ramos',   '2003-01-22'),
('60000000011', 'Paulo Henrique', '1994-12-12');

-- Plano (5 registros)
INSERT INTO Plano (idPlano, nomePlano, valor) VALUES
(1, 'Mensal',      99.90),
```

```

(2,'Trimestral',269.90),
(3,'Semestral', 499.90),
(4,'Anual',      899.90),
(5,'Estudante',  79.90);

-- Modalidade (6 registros). idModalidade e auto_increment; passamos
os
-- valores explicitos para casar com as FKs abaixo.
INSERT INTO Modalidade (idModalidade, nomeModalidade) VALUES
(1,'Musculacao'),
(2,'Crossfit'),
(3,'Pilates'),
(4,'Spinning'),
(5,'Natacao'),
(6,'Funcional');

-- Instrutor (5 registros). Insercao com lista de colunas (cpf,
salario)
-- por causa da coluna dataContratacao adicionada no item (b).
INSERT INTO Instrutor (cpf, salario) VALUES
('11111111111',3500.00),
('22222222222',3200.00),
('33333333333',4100.00),
('44444444444',2900.00),
('55555555555',4500.00);

-- Aluno (11 registros). Note os bairros NULL (Gabriela, Joao,
Nicolas).
INSERT INTO Aluno (cpf, logradouro, numero, bairro, cidade, estado,
cep, idPlano, dataAdesao, dataTermino) VALUES
('60000000001','Rua A', 100,'Centro', 'Lavras',
'MG','37200000',1,'2025-01-10','2025-02-10'),
('60000000002','Rua B', 200, NULL, 'Lavras',
'MG','37200001',2,'2025-02-01','2025-05-01'),
('60000000003','Av C', 300,'Jardim', 'Lavras',
'MG','37200002',3,'2025-03-15','2025-09-15'),
('60000000004','Rua D', 45,'Centro', 'Lavras',
'MG','37200003',4,'2025-01-20','2026-01-20'),
('60000000005','Rua E', 1200, NULL, 'Ijaci',
'MG','37205000',5,'2025-04-01','2025-05-01'),
('60000000007','Av G', 999,'Centro',
'Perdoes','MG','37260000',2,'2025-02-20','2025-05-20'),

```

```

('600000000008','Rua H', 12,'Aeroporto','Lavras',
'MG','37200005',3,'2025-06-01','2025-12-01'),
('600000000009','Rua I', 350, NULL, 'Lavras',
'MG','37200006',4,'2024-12-01','2025-12-01'),
('600000000010','Rua J', 64,'Centro', 'Lavras',
'MG','37200007',5,'2025-03-01','2025-04-01'),
('600000000011','Rua K', 88,'Jardim', 'Lavras',
'MG','37200008',1,'2025-07-01','2025-08-01');

-- Insercao que OMITE cidade e estado para demonstrar os valores
DEFAULT
-- ('Lavras' e 'MG' sao preenchidos automaticamente).
INSERT INTO Aluno (cpf, logradouro, numero, bairro, cep, idPlano,
dataAdesao, dataTermino) VALUES
('600000000006','Rua F', 78,'Vila Nova','37200004',1,'2025-05-
10','2025-06-10');

-- Turma (6 registros). Cada turma referencia uma modalidade e um
instrutor.
INSERT INTO Turma (idTurma, horarioInicio, sala, idModalidade,
cpfInstructor) VALUES
(1,'07:00:00','Sala 1', 1,'11111111111'),
(2,'08:00:00','Sala 2', 2,'22222222222'),
(3,'09:00:00','Sala 3', 3,'33333333333'),
(4,'18:00:00','Sala 1', 4,'44444444444'),
(5,'19:00:00','Piscina',5,'55555555555'),
(6,'06:00:00','Sala 2', 6,'11111111111');

-- Equipamento (8 registros).
INSERT INTO Equipamento (numSerie, nomeEquipamento, idModalidade)
VALUES
('EQ001','Esteira', 1),
('EQ002','Leg Press', 1),
('EQ003','Barra Olimpica', 2),
('EQ004','Bola Suica', 3),
('EQ005','Bike Spinning', 4),
('EQ006','Prancha Natacao', 5),
('EQ007','Kettlebell', 6),
('EQ008','Anilha 10kg', 1);

-- Matricula (13 registros: aluno x turma).
INSERT INTO Matricula (cpfAluno, idTurma, dataMatricula) VALUES

```

```
( '600000000001',1,'2025-01-11'),
( '600000000001',6,'2025-01-12'),
( '600000000002',2,'2025-02-02'),
( '600000000003',3,'2025-03-16'),
( '600000000004',1,'2025-01-21'),
( '600000000004',4,'2025-01-22'),
( '600000000005',5,'2025-04-02'),
( '600000000006',2,'2025-05-11'),
( '600000000007',3,'2025-02-21'),
( '600000000008',6,'2025-06-02'),
( '600000000009',4,'2025-12-02'),
( '600000000010',5,'2025-03-02'),
( '600000000011',1,'2025-07-02');
```

```
-- Verificar tabelas
```

```
SELECT * FROM aluno;
SELECT * FROM equipamento;
SELECT * FROM instrutor;
SELECT * FROM matricula;
SELECT * FROM modalidade;
SELECT * FROM pessoa;
SELECT * FROM plano;
SELECT * FROM turma;
```

d)

```
--
#####
#
-- (d) MODIFICACOES (UPDATE em 5 tabelas + 1 UPDATE aninhado)
--
#####
#

-- UPDATE 1 (Plano): reajuste de 10% no valor do plano Mensal.
UPDATE Plano SET valor = valor * 1.10 WHERE idPlano = 1;

-- UPDATE 2 (Instrutor): aumento fixo de R$200 no salario de um
instrutor.
```



```

UPDATE Instrutor SET salario = salario + 200 WHERE cpf =
'44444444444';

-- UPDATE 3 (Turma): troca a sala da turma 2.
UPDATE Turma SET sala = 'Sala 5' WHERE idTurma = 2;

-- UPDATE 4 (Modalidade): renomeia uma modalidade.
UPDATE Modalidade SET nomeModalidade = 'Musculacao e Forca' WHERE
idModalidade = 1;

-- UPDATE 5 (Aluno) - ANINHADO, ENVOLVENDO MAIS DE UMA TABELA:
-- estende por 12 meses a data de termino dos alunos que contrataram
o
-- plano 'Anual'. A subconsulta busca o idPlano na tabela Plano.
UPDATE Aluno
    SET dataTermino = DATE_ADD(dataAdesao, INTERVAL 12 MONTH)
    WHERE idPlano IN (SELECT idPlano FROM Plano WHERE nomePlano =
'Anual');

```

e)

```

--
#####
#
-- (e) EXCLUSOES (DELETE em 5 tabelas + 1 DELETE aninhado)
--
#####
#

-- DELETE 1 (Matricula): remove uma matricula especifica.
DELETE FROM Matricula WHERE cpfAluno = '60000000010' AND idTurma =
5;

-- DELETE 2 (Equipamento): remove um equipamento pelo numero de
serie.
DELETE FROM Equipamento WHERE numSerie = 'EQ008';

-- DELETE 3 - ANINHADO, ENVOLVENDO MAIS DE UMA TABELA:
-- remove as matriculas das turmas que ocorrem na 'Piscina'.
-- A subconsulta busca os idTurma na tabela Turma.
DELETE FROM Matricula

```

```

WHERE idTurma IN (SELECT idTurma FROM Turma WHERE sala =
'Piscina');

-- DELETE 4 (Turma): apaga a turma 6. Por causa do ON DELETE
CASCADE,
-- as matriculas dessa turma sao removidas automaticamente.
DELETE FROM Turma WHERE idTurma = 6;

-- DELETE 5 (Pessoa): apaga uma pessoa. Como Aluno/Instrutor tem FK
com
-- ON DELETE CASCADE para Pessoa, o aluno correspondente (e suas
-- matriculas) tambem sao removidos em cascata.
DELETE FROM Pessoa WHERE cpf = '60000000010';

```

f)

```

--
#####
#
-- (f) CONSULTAS (12 obrigatorias F1-F12 + extras)
-- Cada consulta tem a descricao do que recupera.
--
#####
#

-- F1 - INNER JOIN: nome do aluno e o plano que ele contratou (com
valor).
SELECT p.nomePessoa, pl.nomePlano, pl.valor
FROM Aluno a
INNER JOIN Pessoa p ON a.cpf = p.cpf
INNER JOIN Plano pl ON a.idPlano = pl.idPlano;

-- F2 - OUTER JOIN: todos os alunos e o id das turmas em que estao
-- matriculados, incluindo os alunos sem nenhuma matricula (idTurma
NULL).
SELECT p.nomePessoa, m.idTurma
FROM Aluno a
JOIN Pessoa p ON a.cpf = p.cpf
LEFT OUTER JOIN Matricula m ON m.cpfAluno = a.cpf
ORDER BY p.nomePessoa, m.idTurma;

```

```

-- F3 - ORDER BY: alunos ordenados da adesao mais recente para a
mais antiga.
SELECT p.nomePessoa, a.dataAdesao
  FROM Aluno a
  JOIN Pessoa p ON a.cpf = p.cpf
  ORDER BY a.dataAdesao DESC, p.nomePessoa ASC;

-- F4 - GROUP BY: quantidade de alunos em cada plano.
SELECT pl.nomePlano, COUNT(*) AS qtdAlunos
  FROM Aluno a
  JOIN Plano pl ON a.idPlano = pl.idPlano
  GROUP BY pl.idPlano, pl.nomePlano
  ORDER BY qtdAlunos DESC;

-- F5 - HAVING: planos que possuem mais de 1 aluno.
SELECT pl.nomePlano, COUNT(*) AS qtdAlunos
  FROM Aluno a
  JOIN Plano pl ON a.idPlano = pl.idPlano
  GROUP BY pl.idPlano, pl.nomePlano
  HAVING COUNT(*) > 1;

-- F6 - UNION: lista unica de pessoas indicando o papel
(Aluno/Instrutor).
SELECT nomePessoa, 'Aluno' AS papel
  FROM Pessoa WHERE cpf IN (SELECT cpf FROM Aluno)
UNION
SELECT nomePessoa, 'Instrutor' AS papel
  FROM Pessoa WHERE cpf IN (SELECT cpf FROM Instrutor)
ORDER BY nomePessoa;

-- F7 - IN: alunos cujo plano e 'Mensal' ou 'Anual'.
SELECT p.nomePessoa
  FROM Aluno a
  JOIN Pessoa p ON a.cpf = p.cpf
  WHERE a.idPlano IN (SELECT idPlano FROM Plano WHERE nomePlano IN
('Mensal', 'Anual'));

-- F8 - LIKE: equipamentos cujo nome comeca com a letra 'B'.
SELECT numSerie, nomeEquipamento
  FROM Equipamento
  WHERE nomeEquipamento LIKE 'B%';

```

```

-- F9 - IS NULL: alunos que nao tem bairro cadastrado.
SELECT p.nomePessoa, a.cidade
  FROM Aluno a
  JOIN Pessoa p ON a.cpf = p.cpf
 WHERE a.bairro IS NULL;

-- F10 - ANY/SOME: planos cujo valor e maior que o de ALGUM dos
planos
-- basicos (Estudante ou Mensal). Equivale a "mais caro que o mais
barato".
SELECT nomePlano, valor
  FROM Plano
 WHERE valor > ANY (SELECT valor FROM Plano WHERE nomePlano IN
('Estudante','Mensal'));

-- F11 - ALL: o(s) plano(s) mais caro(s) - valor maior ou igual a
TODOS.
SELECT nomePlano, valor
  FROM Plano
 WHERE valor >= ALL (SELECT valor FROM Plano);

-- F12 - EXISTS: alunos que possuem pelo menos uma matricula.
SELECT p.nomePessoa
  FROM Aluno a
  JOIN Pessoa p ON a.cpf = p.cpf
 WHERE EXISTS (SELECT 1 FROM Matricula m WHERE m.cpfAluno = a.cpf);

-- F13 (extra - AND / OR): alunos de Lavras com plano Anual OU
Semestral.
SELECT p.nomePessoa, a.cidade, pl.nomePlano
  FROM Aluno a
  JOIN Pessoa p ON a.cpf = p.cpf
  JOIN Plano pl ON a.idPlano = pl.idPlano
 WHERE a.cidade = 'Lavras'
  AND (pl.nomePlano = 'Anual' OR pl.nomePlano = 'Semestral');

-- F14 (extra - BETWEEN): turmas que comecam entre 06:00 e 09:00.
SELECT idTurma, horarioInicio, sala
  FROM Turma
 WHERE horarioInicio BETWEEN '06:00:00' AND '09:00:00'
 ORDER BY horarioInicio;

```

```

-- F15 (extra - NOT IN + LIKE): equipamentos que NAO sao de
Musculacao.
SELECT nomeEquipamento
      FROM Equipamento
     WHERE idModalidade NOT IN (SELECT idModalidade FROM Modalidade
WHERE nomeModalidade LIKE 'Musculacao%');

-- F16 (extra - LEFT JOIN + GROUP BY + agregada): receita potencial
por
-- plano (soma do valor dos alunos de cada plano), incluindo planos
sem aluno.
SELECT pl.nomePlano, COUNT(a.cpf) AS alunos, pl.valor,
       SUM(pl.valor) AS receitaPotencial
   FROM Plano pl
  LEFT JOIN Aluno a ON a.idPlano = pl.idPlano
  GROUP BY pl.idPlano, pl.nomePlano, pl.valor
  ORDER BY receitaPotencial DESC;

```

g)

```

--
#####
#
-- (g) VISOES (Views) - 3 views + exemplo de uso de cada
--
#####
#

-- View 1: alunos com os dados do plano contratado.
DROP VIEW IF EXISTS vw_aluno_plano;
CREATE VIEW vw_aluno_plano AS
  SELECT p.cpf, p.nomePessoa, pl.nomePlano, pl.valor,
         a.dataAdesao, a.dataTermino
    FROM Aluno a
   JOIN Pessoa p ON a.cpf = p.cpf
   JOIN Plano  pl ON a.idPlano = pl.idPlano;
-- Uso da View 1: alunos cujo plano custa mais de R$200.
SELECT * FROM vw_aluno_plano WHERE valor > 200;

-- View 2: detalhe das turmas (modalidade + instrutor).
DROP VIEW IF EXISTS vw_turma_detalhe;
CREATE VIEW vw_turma_detalhe AS

```

```

SELECT t.idTurma, t.horarioInicio, t.sala,
       m.nomeModalidade, pe.nomePessoa AS instrutor
FROM Turma t
JOIN Modalidade m ON t.idModalidade = m.idModalidade
JOIN Instrutor i ON t.cpfInstrutor = i.cpf
JOIN Pessoa pe ON i.cpf = pe.cpf;
-- Uso da View 2: todas as turmas ordenadas por horario.
SELECT * FROM vw_turma_detalhe ORDER BY horarioInicio;

-- View 3: ocupacao das turmas (quantos alunos matriculados em cada
uma).
DROP VIEW IF EXISTS vw_ocupacao_turma;
CREATE VIEW vw_ocupacao_turma AS
  SELECT t.idTurma, m.nomeModalidade, COUNT(mt.cpfAluno) AS
matriculados
  FROM Turma t
  JOIN Modalidade m ON t.idModalidade = m.idModalidade
  LEFT JOIN Matricula mt ON mt.idTurma = t.idTurma
  GROUP BY t.idTurma, m.nomeModalidade;
-- Uso da View 3: turmas que tem pelo menos um aluno matriculado.
SELECT * FROM vw_ocupacao_turma WHERE matriculados >= 1;

```

h)

```

--
#####
#
-- (h) USUARIOS, GRANT E REVOKE
--
#####
#

-- Cria dois usuarios (IF NOT EXISTS deixa o script re-executavel).
CREATE USER IF NOT EXISTS 'recepcao'@'localhost' IDENTIFIED BY
'senha123';
CREATE USER IF NOT EXISTS 'gerente'@'localhost' IDENTIFIED BY
'senha456';

```

```

-- Concede a recepcao permissao de consultar/insserir/atualizar
alunos
-- e de consultar a view de alunos x plano.
GRANT SELECT, INSERT, UPDATE ON academia.Aluno TO
'recepcao'@'localhost';
GRANT SELECT ON academia.vw_aluno_plano TO
'recepcao'@'localhost';

-- Concede ao gerente todos os privilegios sobre o banco academia.
GRANT ALL PRIVILEGES ON academia.* TO 'gerente'@'localhost';

-- Revoga da recepcao a permissao de alterar dados de alunos.
REVOKE UPDATE ON academia.Aluno FROM 'recepcao'@'localhost';

-- Aplica as alteracoes de privilegios.
FLUSH PRIVILEGES;

-- Consulta para conferir as permissoes atuais da recepcao.
SHOW GRANTS FOR 'recepcao'@'localhost';

```

i)

```

--
#####
#
-- (i) PROCEDIMENTOS E FUNCOES (3) - com IF, CASE WHEN, WHILE,
variaveis
--
#####
#

-- FUNCAO 1: classifica um valor de plano em faixa (usa CASE WHEN).
DROP FUNCTION IF EXISTS fn_faixa_valor;
DELIMITER $$
CREATE FUNCTION fn_faixa_valor(p_valor DECIMAL(7,2))
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    DECLARE v_faixa VARCHAR(10);          -- declaracao de variavel
    CASE
        WHEN p_valor < 100 THEN SET v_faixa = 'Baixo';

```

```

        WHEN p_valor < 500 THEN SET v_faixa = 'Medio';
        ELSE
            SET v_faixa = 'Alto';
    END CASE;
    RETURN v_faixa;
END$$
DELIMITER ;
-- Teste da Funcao 1: mostra a faixa de cada plano.
SELECT nomePlano, valor, fn_faixa_valor(valor) AS faixa FROM Plano;

-- PROCEDIMENTO 2: total de matriculas de um aluno (parametro de
SAIDA).
-- Usa IF para tratar aluno inexistente e a funcao agregada COUNT.
DROP PROCEDURE IF EXISTS sp_total_matriculas_aluno;
DELIMITER $$
CREATE PROCEDURE sp_total_matriculas_aluno(IN p_cpf CHAR(11), OUT
p_total INT)
BEGIN
    DECLARE v_existe INT;
    SELECT COUNT(*) INTO v_existe FROM Aluno WHERE cpf = p_cpf;
    IF v_existe = 0 THEN
        SET p_total = -1;      -- -1 sinaliza que o aluno nao existe
    ELSE
        SELECT COUNT(*) INTO p_total FROM Matricula WHERE cpfAluno =
p_cpf;
    END IF;
END$$
DELIMITER ;
-- Teste do Procedimento 2: conta as matriculas do aluno
60000000001.
CALL sp_total_matriculas_aluno('60000000001', @total);
SELECT @total AS total_matriculas;

-- PROCEDIMENTO 3: aplica um reajuste percentual a um plano,
repetido
-- p_vezes (usa IF para validar, WHILE para repetir e variavel
contador).
DROP PROCEDURE IF EXISTS sp_reajuste_iterativo;
DELIMITER $$
CREATE PROCEDURE sp_reajuste_iterativo(IN p_idPlano INT, IN
p_percentual DECIMAL(5,2), IN p_vezes INT)
BEGIN

```



```

        DECLARE v_contador INT DEFAULT 0;      -- variavel de controle do
laco
        DECLARE v_existe  INT;
        SELECT COUNT(*) INTO v_existe FROM Plano WHERE idPlano =
p_idPlano;
        IF v_existe = 0 OR p_percentual <= 0 OR p_vezes <= 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Parametros invalidos ou plano
inexistente';
        ELSE
            WHILE v_contador < p_vezes DO
                UPDATE Plano
                    SET valor = valor * (1 + p_percentual/100)
                    WHERE idPlano = p_idPlano;
                SET v_contador = v_contador + 1;
            END WHILE;
        END IF;
    END$$
DELIMITER ;
-- Teste do Procedimento 3: valor antes, aplica 5% tres vezes, valor
depois.
SELECT valor AS valor_antes FROM Plano WHERE idPlano = 2;
CALL sp_reajuste_iterativo(2, 5, 3);
SELECT valor AS valor_depois FROM Plano WHERE idPlano = 2;

```

```

j)
--
#####
#
-- (j) TRIGGERS (3) - um para INSERT, um para UPDATE, um para DELETE
--
-- Cada trigger tem um proposito DIFERENTE, para cobrir os tres
usos
-- classicos de gatilhos (e nao apenas gerar log):
-- 1. INSERT -> VALIDACAO DE INTEGRIDADE entre tabelas.
-- 2. UPDATE -> REGRA DE NEGOCIO sobre a propria linha.
-- 3. DELETE -> AUDITORIA do evento.
--
-- O trigger de INSERT tambem cobre a correcao pedida na
revisao:
-- a heranca Pessoa -> {Aluno, Instrutor} e uma especializacao

```

```

--      DISJUNTA com cardinalidade 1:1 (cada Pessoa e Aluno OU
Instrutor,
--      nunca os dois). O modelo relacional puro (PK = FK) garante o
"no
--      maximo 1", mas NAO garante a disjuncao; o trigger fecha essa
lacuna.
--
#####
#

-- Tabela de log usada pelo trigger de auditoria (DELETE).
CREATE TABLE IF NOT EXISTS LogAuditoria (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    evento      VARCHAR(20),
    tabela      VARCHAR(30),
    detalhe     VARCHAR(200),
    dataHora    DATETIME
);

--
-----
-
-- TRIGGER 1 (INSERT) - VALIDACAO DE INTEGRIDADE
-- Antes de inserir um Aluno:
-- (a) garante a especializacao DISJUNTA 1:1 da heranca: o mesmo
cpf
--      nao pode ja ser um Instrutor;
-- (b) garante coerencia temporal: dataTermino > dataAdesao.
-- Regras que nem FK nem CHECK conseguem expressar sozinhas.
--
-----
-
DROP TRIGGER IF EXISTS trg_aluno_before_insert;
DELIMITER $$
CREATE TRIGGER trg_aluno_before_insert
BEFORE INSERT ON Aluno
FOR EACH ROW
BEGIN
    -- (a) disjuncao da heranca: se o cpf ja e Instrutor, barra o
INSERT.
    IF EXISTS (SELECT 1 FROM Instrutor WHERE cpf = NEW.cpf) THEN
        SIGNAL SQLSTATE '45000'

```

```
        SET MESSAGE_TEXT = 'Heranca 1:1 disjunta: este cpf ja e
Instrutor e nao pode ser Aluno.';
    END IF;
```

```
    -- (b) coerencia das datas do plano.
    IF NEW.dataTermino <= NEW.dataAdesao THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'dataTermino deve ser posterior a
dataAdesao.';
    END IF;
END$$
DELIMITER ;
```

```
--
-----
-
-- TRIGGER 2 (UPDATE) - REGRA DE NEGOCIO
-- O salario de um instrutor nunca pode ser REDUZIDO por um UPDATE.
-- Aumentos sao permitidos; qualquer reducao e bloqueada.
--
-----
-
DROP TRIGGER IF EXISTS trg_instrutor_before_update;
DELIMITER $$
CREATE TRIGGER trg_instrutor_before_update
BEFORE UPDATE ON Instrutor
FOR EACH ROW
BEGIN
    IF NEW.salario < OLD.salario THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Reducao de salario nao permitida
para instrutores.';
    END IF;
END$$
DELIMITER ;
```

```
--
-----
-
-- TRIGGER 3 (DELETE) - AUDITORIA
-- Registra no log toda exclusao de aluno, guardando quem foi
removido.
```

```

--
-----
-
DROP TRIGGER IF EXISTS trg_aluno_after_delete;
DELIMITER $$
CREATE TRIGGER trg_aluno_after_delete
AFTER DELETE ON Aluno
FOR EACH ROW
BEGIN
    INSERT INTO LogAuditoria (evento, tabela, detalhe, dataHora)
    VALUES ('DELETE','Aluno',
            CONCAT('Aluno removido: ', OLD.cpf,
                    ' (plano ', OLD.idPlano, ')'),
            NOW());
END$$
DELIMITER ;

-- ----- TESTES DOS TRIGGERS -----

-- ===== Trigger 1 (INSERT / validacao) =====

-- 1.1 DISPARA e PERMITE: pessoa nova, apenas Aluno, datas
coerentes -> OK.
INSERT INTO Pessoa (cpf, nomePessoa, dataNasc)
VALUES ('70000000001','Rafael Teixeira','2000-06-15');
INSERT INTO Aluno (cpf, logradouro, numero, bairro, cep, idPlano,
dataAdesao, dataTermino)
VALUES ('70000000001','Rua Nova',
15,'Centro','37200099',1,'2025-09-01','2025-10-01');

-- 1.2 DISPARA e BLOQUEIA (heranca 1:1 disjunta): '1111111111' ja e
-- Instrutor, entao nao pode virar Aluno. Deve gerar erro 45000.
INSERT INTO Aluno (cpf, logradouro, numero, bairro, cep, idPlano,
dataAdesao, dataTermino)
VALUES ('1111111111','Rua X',
10,'Centro','37200098',1,'2025-09-01','2025-10-01');

-- 1.3 DISPARA e BLOQUEIA (datas invalidas): termino antes da
adesao.
INSERT INTO Pessoa (cpf, nomePessoa, dataNasc)
VALUES ('70000000002','Sofia Ramos','1999-02-02');
INSERT INTO Aluno (cpf, logradouro, numero, bairro, cep, idPlano,

```

```

dataAdesao, dataTermino)
VALUES ('70000000002','Rua Y',
20,'Centro','37200097',1,'2025-10-01','2025-09-01');

-- ===== Trigger 2 (UPDATE / regra de negocio) =====

-- 2.1 DISPARA e PERMITE: aumento de salario -> OK.
UPDATE Instrutor SET salario = salario + 100 WHERE cpf =
'11111111111';
-- 2.2 DISPARA e BLOQUEIA: reducao de salario -> erro 45000.
UPDATE Instrutor SET salario = salario - 500 WHERE cpf =
'11111111111';
-- 2.3 NAO DISPARA: alterar outra tabela (Plano) nao aciona este
trigger.
UPDATE Plano SET valor = valor WHERE idPlano = 1;

-- ===== Trigger 3 (DELETE / auditoria) =====

-- 3.1 DISPARA: excluir um aluno gera registro no log.
DELETE FROM Aluno WHERE cpf = '60000000007';
SELECT * FROM LogAuditoria WHERE tabela = 'Aluno';
-- 3.2 NAO DISPARA: delete que nao casa nenhuma linha (cpf
inexistente).
DELETE FROM Aluno WHERE cpf = '00000000000';

-- Visao geral de todos os eventos registrados pelos triggers.
SELECT * FROM LogAuditoria ORDER BY id;

```